Formally Verified Code Generation via Petri Nets for Rust's Ownership Model

Kaiwen Zhang
Guanjun Liu*
zhangkw@tongji.edu.cn
liuguanjun@tongji.edu.cn
Tongji University
Shanghia, Shanghai, China

Figure 1: Seattle Mariners at Spring Training, 2010.

Abstract

The strict static analysis mechanisms of the Rust programming language, particularly its affine type system and ownership model, pose significant challenges for automated test generation. Conventional mutation-based fuzzers often fail to navigate the complex state dependencies of Rust APIs, resulting in a high volume of semantically invalid test cases that are rejected during compilation. To address this, we propose a formal modeling framework that transforms Rust libraries into Colored Petri Nets (CPN), enabling the generation of type-safe API call sequences.

Our approach leverages the `rustdoc-json` intermediate representation to extract precise type definitions and function signatures. We introduce a rigorous mapping strategy where composite data structures are formalized as recursive Color Sets, and API functions are modeled as state transitions. A key contribution of this work is a constraint-based logic system for handling Rust's generic polymorphism and Trait bounds. Instead of topological expansion, which leads to state-space explosion, we model generic constraints as symbolic guards on transitions, ensuring that token flow strictly adheres to the library's type contracts.

The resulting CPN model serves as a formal state-space representation of the library, where reachability analysis yields executable API skeletons. This methodology provides a foundational structure for downstream Large Language Model (LLM) based program synthesis, shifting the fuzzing paradigm from random mutation to model-driven construction.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Rust, Colored Petri Nets, Formal Modeling, Fuzzing, Type Systems, API Sequence Generation

ACM Reference Format:
Kaiwen Zhang and Guanjun Liu. 2018. Formally Verified Code Generation via Petri Nets for Rust's Ownership Model. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Rust has revolutionized systems programming by enforcing memory safety through its ownership and borrowing rules at compile time, eliminating common errors like data races and use-after-free without runtime overhead. The borrow checker, a core component of the Rust compiler (rustc), statically verifies these rules, enabling

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

"fearless concurrency" where developers can write parallel code with confidence in its safety.

Rust has revolutionized systems programming by enforcing memory safety through a strict ownership and borrowing discipline. While the borrow checker empowers developers with "fearless concurrency," it presents a formidable barrier for automated code synthesis and fuzzing. Conventional generative approaches, such as grammar-based fuzzing, act blindly regarding type states, resulting in a vast majority of synthesized programs being rejected by the compiler due to ownership violations. This inefficiency severely limits the scope of automated testing in the Rust ecosystem.

Existing formal foundations for Rust, such as Place Capability Graphs (PCG) and RustBelt, primarily focus on verification—checking the soundness of existing code. While these models provide rigorous abstractions for analyzing validity, they lack the generative mechanisms required for synthesis. There is a critical need for a formal model that can proactively guide the construction of valid code sequences rather than merely rejecting invalid ones.

However, while Rust's type system is powerful for verification, formal models that support *code generation*—synthesizing new, verified code snippets that adhere to these rules—are underdeveloped. Existing works, such as RustBelt [**?**], Stacked Borrows [**?**], and Place Capability Graphs (PCG) [**?**], focus primarily on verifying or analyzing existing code. They provide sound abstractions for borrow checking but lack mechanisms for proactive synthesis, which is crucial for applications like fuzz testing, automated code repair, and program synthesis.

This paper addresses this gap by bridging Linear Logic and Automata Theory. We propose a novel modeling framework that maps Rust's type system to High-Level Petri Nets, exploiting the structural isomorphism between Rust's affine types (resources that are consumed) and Petri Net tokens (flow that is irreversible). Unlike static dependency graphs, our model captures the dynamic lifecycle of resources: ownership transfer is modeled as token consumption, mutable borrowing as loan-return cycles that temporarily block access to owners, and generic instantiation as net unfolding.

This paper addresses this gap by proposing a Petri Net-based formal model for Rust's ownership and borrowing rules. Petri Nets, a classical formalism for modeling concurrent and distributed systems, offer natural advantages in simulating dynamic behaviors through token flows and transitions. We customize the net with labeled places and transitions to precisely encode Rust's key concepts: ownership transfers as token movements, borrowing as temporary token allocations, and lifetimes as scoped invariants.

We present a pipeline that automatically extracts API semantics from cargo doc metadata to construct a Rust-specific Petri Net. We formulate the code generation task as a Reachability Problem within this net. By traversing valid firing sequences in the Vector Addition System, our algorithm produces API sequences that are correct-by-construction regarding ownership and lifetime constraints. Special attention is given to handling complex features such as non-lexical lifetimes (NLL), trait bounds, and structural decomposition of composite types.

Our contributions are threefold:

- A formal definition of a Labeled Petri Net tailored to Rust's type system, including custom firing rules that enforce AXM and handle non-lexical lifetimes (NLL), composite types, and generics.
- An algorithm for generating borrow-safe Rust code from reachable markings in the net, leveraging reachability analysis to enumerate valid paths.
- Empirical evaluations on top Rust crates (e.g., cargo, tokio), demonstrating that our approach generates 20-30% more valid fuzz inputs than random methods, with applications in discovering type-related bugs.

By shifting from verification to synthesis, our model not only complements existing tools like PCG but also opens new avenues for automated testing in Rust ecosystems. The implementation is open-source and available at [GitHub link].

2 Background and Problem Formulation

To understand the challenge of synthesizing valid Rust code, one must first grasp the strict static analysis performed by the Rust compiler. Unlike garbage-collected languages (e.g., Java, Python) or manually managed ones (e.g., C/C++), Rust employs an *affine type system* and a *borrow checker* to guarantee memory safety.

2.1 Ownership and Affine Types

At the core of Rust is the concept of *ownership*. Every value has a single owner, and when the owner goes out of scope, the value is dropped. Crucially, Rust enforces *affine semantics* for non-primitive types: a value can be used (moved) at most once.

Consider the code in Listing 1. Assigning `s1` to `s2` transfers ownership. Any subsequent attempt to access `s1` is flagged as a compile-time error.

```rust
fn main() {
    let s1 = String::from("resource");
    let s2 = s1; // Ownership moved to s2

    // Compile Error: borrow of moved value: `s1`
    println!("{}", s1);
}
```

Listing 1: Ownership transfer causing a "use-after-move" error.

Implication for Synthesis: A naive fuzzer unaware of these semantics might generate sequences that reuse variables after they have been moved. **Connection to Our Model:** This aligns perfectly with the token consumption mechanics of Petri Nets. A token in a Petri Net represents a resource; once a transition (function) consumes a token (variable), it is removed from the place, rendering it unavailable for future transitions unless explicitly regenerated.

2.2 The Borrowing Discipline: Aliasing XOR Mutability

Rust allows temporary access to values via references (borrowing). The borrow checker enforces the *Aliasing XOR Mutability* rule: at any given point, a variable can have either multiple immutable references (`&T`) OR exactly one mutable reference (`&mut T`), but not

both. Furthermore, while a mutable reference exists, the original owner is effectively "frozen."

Listing 2 demonstrates a violation where the original owner `data` is accessed while a mutable borrow `borrow` is still active.

```rust
fn main() {
    let mut data = vec![1, 2, 3];
    let borrow = &mut data; // Mutable borrow
        starts

    // Compile Error: cannot use `data` because
    // it was mutably borrowed
    data.push(4);

    println!("{:?}", borrow); // Borrow ends here
}
```

Listing 2: Violation of the exclusive mutability rule.

Implication for Synthesis: Valid code generation requires tracking the complex "locking" state of every variable. **Connection to Our Model:** We model this statefulness using the network topology. A mutable borrow is represented as a *loan-return cycle*: the token representing `data` is temporarily removed from its place (locking it) and is only returned when the transition representing the end of the borrow fires.

2.3 Lifetimes and Scoping

Lifetimes are named regions of code that ensure references do not outlive the data they point to. The compiler constructs a control-flow graph to verify that any data referenced by `&'a T` is valid for at least lifetime `'a`.

Listing 3 illustrates a "dangling pointer" scenario, which Rust prevents statically.

```rust
fn main() {
    let r;
    {
        let x = 5;
        r = &x; // Error: `x` does not live long
            enough
    } // `x` is dropped here
    println!("{}", r);
}
```

Listing 3: A reference outliving its referent.

Connection to Our Model: In our Petri Net, scopes are not merely syntactic blocks but are encoded as causal dependencies between transitions. A "Drop" transition for a resource is strictly guarded; it cannot fire if there are outstanding "Reference" tokens circulating in the network, ensuring lifetime invariants are respected by construction.

2.4 The Gap: Verification vs. Synthesis

While formal models like RustBelt [?] and Place Capability Graphs (PCG) provide a solid foundation for *verifying* the safety of existing code, they are not optimized for *generating* new code.

A random fuzzer operating on the grammar level has a near-zero probability of guessing a sequence of operations that satisfies ownership, borrowing, and lifetime constraints simultaneously. This necessitates a generative model that is *correct-by-construction*. By mapping Rust's type system to a Petri Net, we transform the problem of satisfying these constraints into a graph reachability problem, ensuring that every generated path corresponds to a valid Rust program.

3 Methodology

This section formalizes the transformation framework. We first provide an abstract definition of a Rust library derived from the static analysis of `rustdoc-json`. Subsequently, we define the target formalism, Colored Petri Nets (CPN). Finally, we define the mapping function Ψ that transforms the static library definition into a dynamic CPN model executable for fuzzing sequence generation.

3.1 Formal Abstraction of Rust Libraries

We abstract a Rust library (Crate) as a tuple representing its type system and interface surface.

Definition 3.1 (Rust Library Model). *A Rust library is defined as a tuple $\mathcal{L} = (\mathcal{U}, \mathcal{F}, \mathcal{K}, \mathcal{I})$, where:*

- *$\mathcal{U} = \mathcal{U}_{prim} \cup \mathcal{U}_{comp}$ is the universe of types, consisting of primitive types (e.g., `i32`, `bool`) and composite types (Structs, Enums).*
- *$\mathcal{F}$ is the set of public API functions. Each $f \in \mathcal{F}$ is characterized by a signature $sig(f) : \mathcal{U}^* \to \mathcal{U}$.*
- *$\mathcal{K}$ is the set of Traits (interfaces), representing behavioral constraints.*
- *$\mathcal{I} \subseteq \mathcal{U} \times \mathcal{K}$ is the implementation relation. We write $\tau \vdash \kappa$ if type $\tau \in \mathcal{U}$ implements trait $\kappa \in \mathcal{K}$.*

Definition 3.2 (Composite Structure). *For any composite type $\tau \in \mathcal{U}_{comp}$:*

- *If τ is a **Struct**, it is defined by an ordered set of fields $S_\tau = \{(id_1, \tau_1), \dots, (id_n, \tau_n)\}$.*
- *If τ is an **Enum**, it is defined by a set of variants $E_\tau = \{v_1(\tau_{v1}), \dots, v_m(\tau_{vm})\}$.*

3.2 Formal Definition of Colored Petri Nets

We utilize the formal definition of Colored Petri Nets as proposed by Jensen to model the concurrent behavior and state of the API usage.

Definition 3.3 (Colored Petri Net). *A CPN is a 9-tuple $\mathcal{N} = (\Sigma, P, T, A, N, C, G, E, I)$, where:*

(1) *Σ is a finite, non-empty set of non-empty **Color Sets**.*
(2) *P is a finite set of **Places**.*
(3) *T is a finite set of **Transitions**, such that $P \cap T = \emptyset$.*
(4) *A is a finite set of **Arcs**, such that $P \cap A = T \cap A = \emptyset$.*
(5) *$N : A \to (P \times T) \cup (T \times P)$ is the **Node Function**, defining the source and destination of arcs.*
(6) *$C : P \to \Sigma$ is the **Color Function**, assigning a color set to each place.*
(7) *$G : T \to \mathbb{B}$ is the **Guard Function**, mapping each transition to a boolean expression.*
(8) *$E : A \to Expr$ is the **Arc Expression Function**, evaluating to a multi-set of token colors.*
(9) *$I : P \to Expr_{closed}$ is the **Initialization Function**, defining the initial marking.*

3.3 Mapping Strategy: Rust to CPN

The core of our methodology is the mapping function $\Psi : \mathcal{L} \to \mathcal{N}$, which translates the static semantics of $\mathcal{L}$ into the dynamic topology of $\mathcal{N}$.

3.3.1 Type Mapping (Σ and C). The first step transforms Rust memory layouts into CPN Color Sets (Σ). To support runtime polymorphism simulation, we define a universal token structure.

Definition 3.4 (Token Structure). *Every token $\omega \in \Sigma_{univ}$ is a tuple $\langle \texttt{tid}, \texttt{val} \rangle$, where $\texttt{tid} \in \mathcal{ID}$ uniquely identifies the concrete type in $\mathcal{U}$, and $\texttt{val}$ holds the structural data.*

Mapping Rule 3.1 (Data Structure Transformation). *Let $\phi : \mathcal{U} \to \Sigma$ be the type transformation function:*

1. ***Primitives**: For $b \in \mathcal{U}_{prim}$, $\phi(b)$ maps directly to standard CPN types (e.g., Integer, Boolean, String).*
2. ***Structs**: For a struct τ with fields types $\tau_1, \dots, \tau_n$, $\phi(\tau)$ is defined as a **Product Color Set**:*

$$\phi(\tau) = \prod_{i=1}^{n} \phi(\tau_i) = \phi(\tau_1) \times \phi(\tau_2) \times \cdots \times \phi(\tau_n) \tag{1}$$

   This preserves the aggregation relationship of the struct members.
3. ***Enums**: For an enum τ with variants $v_1, \dots, v_m$, $\phi(\tau)$ is defined as a **Disjoint Union Color Set** (Tagged Union):*

$$\phi(\tau) = \coprod_{j=1}^{m} (\texttt{tag}_{v_j} \times \phi(content(v_j))) \tag{2}$$

Based on this, the set of Places P is constructed. For every visible type $\tau \in \mathcal{U}$ appearing in public API signatures, we create a unique place $p_\tau \in P$, such that $C(p_\tau) = \phi(\tau)$.

3.3.2 Function Mapping (T, A, and E). Transitions model the execution of functions. This mapping ensures that the flow of tokens adheres to Rust's ownership and borrowing rules (simplified for reachability analysis).

Mapping Rule 3.2 (Function Transformation). *For each function $f \in \mathcal{F}$ with signature $f(x_1 : \tau_1, \dots, x_k : \tau_k) \to \tau_{ret}$:*

1. *Create a transition $t_f \in T$.*
2. ***Input Arcs**: For each argument x_i, create an arc $a_i \in A$ from p_{τ_i} to t_f. The arc expression $E(a_i)$ binds the variable name x_i.*
3. ***Output Arcs**: Create an arc $a_{out} \in A$ from t_f to $p_{\tau_{ret}}$. The expression $E(a_{out})$ represents the function result.*

3.3.3 Constraint Mapping (G). The crucial innovation of our approach is mapping generic bounds to CPN Guard expressions, converting type-checking problems into constraint satisfaction problems during net execution.

Mapping Rule 3.3 (Generics and Trait Bounds). *Let f be a generic function with type parameter T constrained by trait bounds $\{\kappa_1, \dots, \kappa_r\} \subseteq \mathcal{K}$. The Guard function $G(t_f)$ is constructed as:*

$$G(t_f) = \lambda\omega. \bigwedge_{j=1}^{r} (\omega.\texttt{tid} \vdash \kappa_j) \tag{3}$$

where $\omega.\texttt{tid}$ extracts the type identifier from the incoming token. The relation $\vdash$ is evaluated against the pre-computed implementation registry $\mathcal{I}$.

This mapping ensures that the transition t_f is enabled if and only if the concrete data types flowing into the function satisfy the Rust compiler's requirements for the generic parameters.

Acknowledgments

To Robert, for the bagels and explaining CMYK and color spaces.

References

A Research Methods

A.1 Part One

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna, id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

A.2 Part Two

Etiam commodo feugiat nisl pulvinar pellentesque. Etiam auctor sodales ligula, non varius nibh pulvinar semper. Suspendisse nec lectus non ipsum convallis congue hendrerit vitae sapien. Donec at laoreet eros. Vivamus non purus placerat, scelerisque diam eu, cursus ante. Etiam aliquam tortor auctor efficitur mattis.

B Online Resources

Nam id fermentum dui. Suspendisse sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

Nam interdum magna at lectus dignissim, ac dignissim lorem rhoncus. Maecenas eu arcu ac neque placerat aliquam. Nunc pulvinar massa et mattis lacinia.